AI INTERACTION LOG – Sprint 2

Title: Implement Job Application Submission and Status Tracking API

Purpose of AI Use: Database migration design, Express.js route implementation, and unit test debugging.

Prompt/Response: Prompts used: "Help design a PostgreSQL migration script for a job applications table in an Express.js backend that enforces valid statuses (Applied, Under Review, Interview, Offered, Rejected) and prevents duplicate applications per user", plus debugging prompts for pg-mem test runner integration.

AI-Suggested Content: Provided the SQL schema structure using SERIAL PRIMARY KEY, foreign keys, CHECK constraints on valid status strings, Express routes (POST /api/applications, GET /api/applications/me, PATCH /api/applications/:id/status), and automated test cases using Node's native test runner (node --test).

Validation: Tested the implementation locally using npm test. Confirmed that the test suite runs successfully with pass 1 and fail 0, validating schema migrations, constraints, status filters, and duplicate application handling.

Reflection: The AI assistance was highly efficient for structuring relational database constraints correctly and establishing robust Express routing and endpoint testing without manual documentation lookup.

Responsible Person: Jack Wu

Task ID/Title: Sprint 2 Backend Job Application Submission & Tracking API Implementation and Testing

Purpose of AI Use: Coding, debugging, testing assistance, and git merge conflict resolution guidance

Chat Link or Prompt/Response: “There has been a git hub merge conflict since I didn’t merge my code into the repo. How should I proceed with this to fix this conflict?”

AI-Suggested Content: Guidance on resolving Git merge conflicts with team authentication updates, and changing the code in the review to make it work while keeping the original code without making fundamental changes.

Validation: Verified code changes locally by running automated integration test suites (npm test) resulting in clean execution (pass 1, fail 0) and manually executing requests via the VS Code REST Client to confirm correct HTTP status codes and JSON responses.

Decision: Modified before use (adjusted code blocks to match local database schemas, existing repository structures, and JWT authentication middleware integration).

Reflection: The AI assistance was useful in structuring backend validation logic, debugging module loading errors, and streamlining manual testing procedures without requiring external GUI tools.

Responsible Person: Jack Wu

Task ID/Title: Guidance on Testing Implementation via VS Code REST Client

Purpose of AI Use: Troubleshooting and testing workflow guidance. Thunder Client wasn't working properly, so an alternative application was needed to perform testing on the APIs.

Chat Link or Prompt/Response: Prompts used: "Can you please help me perform a test? Thunder Client is currently asking for registration, and I am too lazy to do that. Please give me other alternative methods to perform a test on the API of the application process.

AI-Suggested Content: Provided instructions restrictions by using VS Code's built-in REST Client extension (test.http), along with a structured template of HTTP requests for testing registration, JWT login authentication, job application submission (POST), retrieval (GET), status updates (PATCH), and filtering. Gave a basic template on how to create a test.http along with testing with registered user token followed by the process for job application.

Validation: Verified the testing workflow locally by sending HTTP requests through the REST Client extension against the running server, confirming expected responses (201 Created, 409 Conflict, 200 OK).

Decision: Modified before use (adapted the. http request template and variable tokens to fit local user credentials and project routes).

Reflection: The AI guidance was useful in establishing a clean, file-based testing process directly inside the repository without relying on external GUI tools that require account creation.

Responsible Person: Jack Wu